

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по индивидуальной практической работе № 1

по дисциплине
«ИТиВП»

на тему «Разработка безопасного *REST API*»

Выполнил:

студент группы 320601
А. С. Образцов

Проверила:

Н. В. Хаджинова

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью индивидуальной практической работы является разработка безопасного *REST API* для курсового проекта — сервиса планирования задач по методу *GTD* (*Getting Things Done*) с контекстами и напоминаниями (вариант 27). В работе реализуются аутентификация по *JWT*, ролевая модель *user*, *moderator* и *admin*, обязательные меры защиты от типовых угроз *OWASP* (*SQL*-инъекции, перебор паролей, *XSS*, массовые запросы) и десять дополнительных мер, выбранных под особенности предметной области.

Метод *GTD* предполагает, что все дела человека сначала собираются во входящие, а затем разбираются по спискам. Поэтому основной сущностью сервиса является задача, которая в каждый момент находится в одном из шести статусов:

- *inbox* — входящая, ещё не разобранный задача;
- *next* — следующее конкретное действие;
- *waiting* — задача ожидает результата от другого человека;
- *scheduled* — задача привязана к дате, поле *dueDate* обязательно;
- *someday* — когда-нибудь, может быть;
- *done* — выполненная задача.

Контекст описывает место или средство, в котором задачу можно выполнить: *@home*, *@work*, *@calls*. Каждый пользователь ведёт собственный набор контекстов, а контексты, отмеченные как публичные, попадают в общий каталог готовых названий. Напоминание задаётся моментом *remindAt*: маршрут */tasks/reminders/due* возвращает невыполненные задачи владельца, время напоминания которых уже наступило.

Задачи и напоминания являются личными данными: по ним можно восстановить расписание дня, круг общения и планы человека. Поэтому главное требование к *API* — пользователь не должен ни прочитать, ни изменить чужие данные, а учётная запись должна быть защищена от подбора пароля и кражи сессии.

2 ХОД РАБОТЫ

2.1 Архитектура и стек технологий

Сервис написан на *Node.js* с фреймворком *Express 5*. Данные хранятся в *PostgreSQL 16*, запущенной в контейнере *Docker*, доступ к базе выполняется через *ORM Sequelize*. Пароли хешируются библиотекой *bcrypt*, токены подписываются библиотекой *jsonwebtoken*, входные данные проверяются схемами *Joi*. Для защиты используются *helmet*, *cors*, *express-rate-limit* и *sanitize-html*, для журналирования — *winston* с модулем *winston-daily-rotate-file*.

Код разделён на слои: маршруты (*routes*) описывают цепочку *middleware*, контроллеры (*controllers*) читают проверенные данные запроса и вызывают сервисы (*services*), в которых сосредоточена бизнес-логика и проверка прав доступа. Модели (*models*) описывают таблицы и связи между ними. Порядок подключения глобальных *middleware* задан в фабрике приложения:

```
function createApp() {  
  const app = express();  
  
  app.set('trust proxy', config.trustProxy);  
  app.set('query parser', 'simple');  
  app.set('json escape', true);  
  app.set('x-powered-by', false);  
  
  app.use(requestLogger);  
  app.use(helmet(API_SECURITY_HEADERS));  
  app.use(corsMiddleware);  
  app.use(globalLimiter);  
  app.use(requireJson);  
  app.use(express.json({ limit: config.bodyLimit,  
    strict: true }));  
  app.use(cookieParser());  
  app.use(sanitizeBody);  
  
  app.use(routes);  
  app.use(notFoundHandler);  
  app.use(errorHandler);  
  return app;  
}
```

Порядок выбран так, чтобы дешёвые проверки отсекали запрос раньше дорогих: запрос с чужого *Origin* или сверх лимита отклоняется до разбора тела, а тело в формате, отличном от *JSON*, отклоняется до чтения из сокета.

Исходный код размещён в репозитории <https://github.com/Alexandr-Obraztsov/gtd-task-service>, ветка *ipr21*. Там же находится коллекция *Postman* со всеми сценариями проверки: https://github.com/Alexandr-Obraztsov/gtd-task-service/blob/ipr21/postman/gtd-secure-api.postman_collection.json.

2.2 Схема базы данных

База данных состоит из четырёх таблиц. Таблица *users* хранит учётные записи: *email*, хеш пароля, роль, счётчик неудачных входов и время окончания блокировки. Таблица *refresh_tokens* хранит выданные *refresh*-токены в виде *SHA-256* хеша вместе со сроком действия, временем отзыва и хешем токена, который его заменил. Таблицы *contexts* и *tasks* хранят контексты и задачи, у каждой записи есть владелец *ownerId*. Схема базы данных представлена на рисунке 2.1.

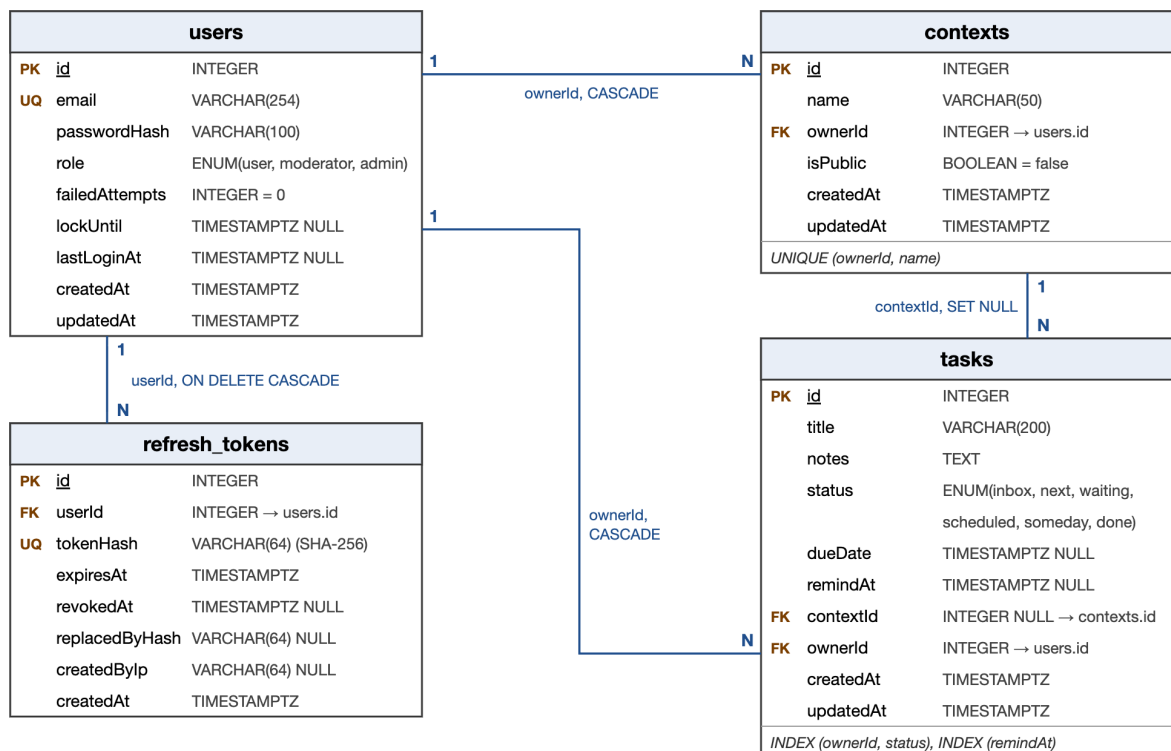


Рисунок 2.1 – Схема базы данных сервиса

При удалении пользователя каскадно удаляются его задачи, контексты и *refresh*-токены. При удалении контекста задачи не удаляются: поле *contextId* у них становится пустым.

2.3 Модели и маршруты

Модель пользователя переопределяет метод *toJSON* и возвращает только открытые поля, поэтому хеш пароля, счётчик попыток и время блокировки не могут попасть в ответ *API* даже по ошибке в контроллере:

```
const PUBLIC_FIELDS = Object.freeze(
  ['id', 'email', 'role', 'createdAt', 'updatedAt']);

class User extends Model {
  isLocked(now = new Date()) {
    return Boolean(this.lockUntil && this.lockUntil > now);
  }

  toJSON() {
    const values = this.get({ plain: true });
    return Object.fromEntries(
      PUBLIC_FIELDS.map((field) => [field, values[field]]));
  }
}

User.init({
  id: { type: DataTypes.INTEGER, autoIncrement: true,
    primaryKey: true },
  email: { type: DataTypes.STRING(254), allowNull: false,
    unique: true, validate: { isEmail: true } },
  passwordHash: { type: DataTypes.STRING(100),
    allowNull: false },
  role: { type: DataTypes.ENUM(...ROLE_VALUES),
    allowNull: false, defaultValue: ROLES.USER },
  failedAttempts: { type: DataTypes.INTEGER,
    allowNull: false, defaultValue: 0 },
  lockUntil: { type: DataTypes.DATE, allowNull: true },
  lastLoginAt: { type: DataTypes.DATE, allowNull: true },
}, { sequelize, modelName: 'User', tableName: 'users' });
```

Модель задачи ограничивает статус перечислением *GTD* и задаёт индексы для выборки по владельцу и статусу и для поиска наступивших напоминаний:

```
Task.init({
  id: { type: DataTypes.INTEGER, autoIncrement: true,
    primaryKey: true },
  title: { type: DataTypes.STRING(200), allowNull: false },
```

```

    notes: { type: DataTypes.TEXT, allowNull: false,
      defaultValue: '' },
    status: { type: DataTypes.ENUM(...TASK_STATUS_VALUES),
      allowNull: false, defaultValue: TASK_STATUS.INBOX },
    dueDate: { type: DataTypes.DATE, allowNull: true },
    remindAt: { type: DataTypes.DATE, allowNull: true },
    contextId: { type: DataTypes.INTEGER, allowNull: true },
    ownerId: { type: DataTypes.INTEGER, allowNull: false },
  }, {
    sequelize, modelName: 'Task', tableName: 'tasks',
    indexes: [{ fields: ['ownerId', 'status'] },
      { fields: ['remindAt'] }],
  });

```

Связи между моделями объявлены в одном месте:

```

const CASCADE = { onDelete: 'CASCADE' };

User.hasMany(RefreshToken, { foreignKey: 'userId',
  as: 'refreshTokens', ...CASCADE });
User.hasMany(Context, { foreignKey: 'ownerId',
  as: 'contexts', ...CASCADE });
User.hasMany(Task, { foreignKey: 'ownerId',
  as: 'tasks', ...CASCADE });
Context.hasMany(Task, { foreignKey: 'contextId',
  as: 'tasks', onDelete: 'SET NULL' });

```

Маршруты аутентификации (файл `src/routes/auth.routes.js`) защищены отдельным ограничителем частоты и схемами проверки входных данных:

```

const router = Router();

router.post('/register', authLimiter,
  validate(schemas.register), controller.register);
router.post('/login', authLimiter,
  validate(schemas.login), controller.login);
router.post('/refresh', authLimiter,
  validate(schemas.refresh), controller.refresh);
router.post('/logout', validate(schemas.logout),
  controller.logout);
router.get('/me', authenticate, validate(schemas.me),
  controller.me);

```

Маршруты задач требуют аутентификации для всех операций, а каждый маршрут имеет собственную схему проверки входных данных. Права на чужую

задачу проверяются в сервисе, поскольку для этого нужна сама запись из базы данных:

```
const router = Router();

router.use(authenticate);

router.get('/', validate(schemas.list), controller.list);
router.post('/', validate(schemas.create),
  controller.create);
router.get('/reminders/due',
  validate(schemas.dueReminders), controller.dueReminders);
router.get('/:id', validate(schemas.getOne),
  controller.getOne);
router.patch('/:id', validate(schemas.update),
  controller.update);
router.delete('/:id', validate(schemas.remove),
  controller.remove);
```

Маршруты управления пользователями дополнительно закрыты проверкой роли: просмотр доступен модератору, изменение роли и удаление — только администратору:

```
router.use(authenticate);

router.get('/', roleGuard(ROLES.MODERATOR),
  validate(schemas.list), controller.list);
router.get('/:id', roleGuard(ROLES.MODERATOR),
  validate(schemas.getOne), controller.getOne);
router.patch('/:id/role', roleGuard(ROLES.ADMIN),
  validate(schemas.changeRole), controller.changeRole);
router.delete('/:id', roleGuard(ROLES.ADMIN),
  validate(schemas.remove), controller.remove);
```

Права на ресурсы, которые зависят от владельца, описаны таблицей: чтение чужого ресурса разрешено с роли *moderator*, удаление — с роли *admin*, изменение чужого ресурса не разрешено никому:

```
const PRIVILEGED_ROLE_BY_ACTION = Object.freeze({
  [ACTIONS.READ]: ROLES.MODERATOR,
  [ACTIONS.UPDATE]: null,
  [ACTIONS.DELETE]: ROLES.ADMIN,
});
```

```
function ensureCanAccess(actor, resource, action, type) {
  if (resource.ownerId === actor.id
    || isPrivileged(actor, action)) {
    return;
  }
  reportForeignAccess(actor, { resourceType: type,
    resourceId: resource.id, ownerId: resource.ownerId,
    action });
  throw new ForbiddenError('Нет доступа к чужому ресурсу',
    'FOREIGN_RESOURCE');
}
```

2.4 Ключевые *middleware*

Middleware authenticate проверяет формат заголовка *Authorization*, подпись и срок действия *access*-токена, а затем загружает пользователя из базы данных. Роль берётся из базы, а не из токена, поэтому понижение роли или удаление учётной записи действуют сразу, не дожидаясь истечения токена:

```
const BEARER_PATTERN =
  /^Bearer ([A-Za-z0-9\-\_]+\.[A-Za-z0-9\-\_]+\.[A-Za-z0-9\-\_]+)$/;

function decodeAccessToken(req, token) {
  try {
    return verifyAccessToken(token);
  } catch (error) {
    if (isTokenExpiredError(error)) {
      throw new UnauthorizedError('Срок действия токена истёк',
        'TOKEN_EXPIRED');
    }
    reportSuspiciousRequest(SEcurity_EVENTS.INVALID_TOKEN,
      req, { reason: error.message });
    throw new UnauthorizedError(
      'Недействительный токен доступа', 'TOKEN_INVALID');
  }
}
```

```
async function authenticate(req, res, next) {
  const payload = decodeAccessToken(req,
    extractBearerToken(req));
  const user = await User.findByPk(Number(payload.sub));
  if (!user) {
    reportSuspiciousRequest(SEcurity_EVENTS.INVALID_TOKEN,
      req, { reason: 'user_not_found', sub: payload.sub });
    throw new UnauthorizedError('Пользователь не найден',

```

```

    'TOKEN_INVALID');
  }
  req.actor = { id: user.id, email: user.email,
    role: user.role, ip: req.ip,
    request: { method: req.method, path: req.originalUrl,
      userAgent: req.get('user-agent') } };
  next();
}

```

Проверка подписи выполняется с явным списком алгоритмов, издателем, аудиторией и типом токена, поэтому *refresh*-токен не может быть использован как *access*-токен, а токен с алгоритмом *none* не будет принят:

```

function verifyToken(token, secret, expectedType) {
  const payload = jwt.verify(token, secret, {
    algorithms: [SIGNING_ALGORITHM],
    issuer: config.jwt.issuer,
    audience: config.jwt.audience,
  });
  if (payload.type !== expectedType) {
    throw new jwt.JsonWebTokenError('unexpected token type');
  }
  return payload;
}

```

Middleware roleGuard принимает минимальную требуемую роль и сравнивает ранги ролей *user* (1), *moderator* (2) и *admin* (3). Отказ записывается в журнал безопасности:

```

const ROLE_RANK = Object.freeze({
  [ROLES.USER]: 1, [ROLES.MODERATOR]: 2, [ROLES.ADMIN]: 3,
});

function hasRole(actualRole, requiredRole) {
  return (ROLE_RANK[actualRole] || 0)
    >= ROLE_RANK[requiredRole];
}

function roleGuard(requiredRole) {
  return (req, res, next) => {
    if (!hasRole(req.actor.role, requiredRole)) {
      reportSuspiciousRequest(
        SECURITY_EVENTS.INSUFFICIENT_ROLE, req,
        { role: req.actor.role, requiredRole });
    }
  };
}

```

```

    throw new ForbiddenError(
      'Операция доступна только для роли ${requiredRole} и выше',
      'INSUFFICIENT_ROLE');
  }
  next();
};
}

```

Middleware validate проверяет все три источника данных запроса: *params*, *query* и *body*. Если для источника схема не задана, он обязан быть пустым, поэтому лишний параметр в строке запроса тоже приводит к ошибке 400. Неизвестные поля тела дополнительно фиксируются как попытка *mass assignment*:

```

const REQUEST_SOURCES = Object.freeze(
  ['params', 'query', 'body']);

const JOI_OPTIONS = Object.freeze({
  abortEarly: false,
  convert: true,
  allowUnknown: false,
  messages: RUSSIAN_MESSAGES,
  errors: { wrap: { label: '' } },
});

function validate(schemas = {}) {
  return (req, res, next) => {
    const validated = {};
    const details = [];
    for (const source of REQUEST_SOURCES) {
      const schema = schemas[source] || emptyObject;
      const { value, error } = schema.validate(
        req[source] ?? {}, JOI_OPTIONS);
      if (error) {
        details.push(...toDetails(source, error));
      }
      validated[source] = value;
    }
    if (details.length) {
      reportUnknownBodyFields(req, details);
      throw new ValidationError('Некорректные данные запроса',
        details.map(({ type, ...rest }) => rest));
    }
    req.validated = validated;
    next();
  };
}

```

```
};
}
```

Ограничение частоты запросов построено на *express-rate-limit*. Глобальный лимитер допускает 300 запросов за 15 минут с одного IP-адреса. Лимитер маршрутов */auth/login*, */auth/register* и */auth/refresh* считает только неуспешные запросы и допускает 20 таких запросов за то же окно, поэтому обычный вход не расходует лимит, а перебор пароля упирается в него быстро:

```
const windowMs = config.rateLimit.windowMinutes * 60 * 1000;
```

```
function createLimiter({ name, limit,
  skipSuccessfulRequests = false }) {
  return rateLimit({
    windowMs,
    limit,
    skipSuccessfulRequests,
    standardHeaders: 'draft-7',
    legacyHeaders: false,
    handler: (req, res, next) => {
      reportSuspiciousRequest(
        SECURITY_EVENTS.RATE_LIMIT_EXCEEDED, req,
        { limiter: name, limit });
      next(new TooManyRequestsError());
    },
  });
}
```

```
const globalLimiter = createLimiter({ name: 'global',
  limit: config.rateLimit.globalMax });
const authLimiter = createLimiter({ name: 'auth',
  limit: config.rateLimit.authMax,
  skipSuccessfulRequests: true });
```

Middleware requireJson отклоняет тело в формате, отличном от *application/json*, ещё до его разбора:

```
const BODY_METHODS = new Set(['POST', 'PUT', 'PATCH']);
```

```
function hasBody(req) {
  return req.get('transfer-encoding') !== undefined
    || Number(req.get('content-length')) > 0;
}
```

```
function requireJson(req, res, next) {
  if (BODY_METHODS.has(req.method) && hasBody(req)
    && !req.is('application/json')) {
    throw new UnsupportedMediaTypeError();
  }
  next();
}
```

2.5 Регистрация, вход и защищённые маршруты

Все запросы в этом и следующих подразделах выполнены к работающему серверу по адресу *http://localhost:3000*, база *PostgreSQL* запущена в *Docker*. *JWT* в снимках сокращены до первых 20 символов, пароли рабочих учётных записей скрыты.

Регистрация принимает только *email* и пароль, роль нового пользователя всегда *user*. Регистрация представлена на рисунке 2.2.

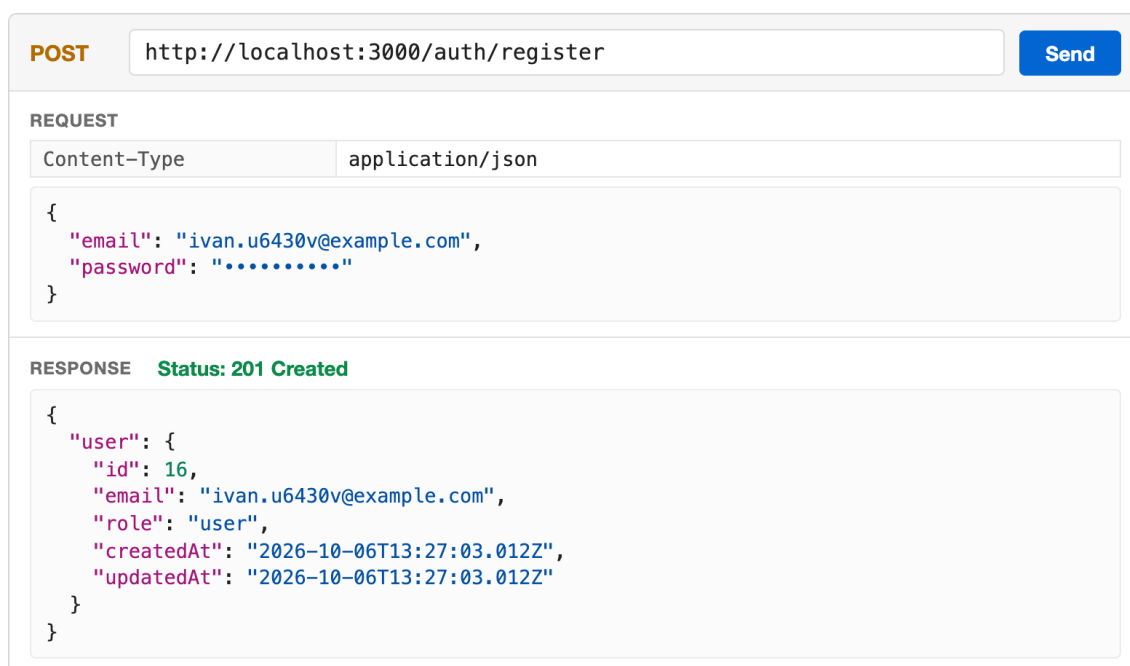


Рисунок 2.2 – Регистрация нового пользователя

При входе сервер возвращает короткоживущий *access*-токен в теле ответа, а *refresh*-токен устанавливает в *cookie* с флагами *HttpOnly* и *SameSite=Strict*, ограниченную путём */auth*. Вход представлен на рисунке 2.3.

POST Send

REQUEST

Content-Type	application/json
--------------	------------------

```
{
  "email": "ivan.u6430v@example.com",
  "password": "....."
}
```

RESPONSE **Status: 200 OK**

Set-Cookie	refreshToken=eyJhbGciOiJIUzI1NiIs...; Max-Age=604800; Path=/auth; Expires=Tue, 13 Oct 2026 13:27:03 GMT; HttpOnly; SameSite=Strict
------------	--

```
{
  "user": {
    "id": 16,
    "email": "ivan.u6430v@example.com",
    "role": "user",
    "createdAt": "2026-10-06T13:27:03.012Z",
    "updatedAt": "2026-10-06T13:27:03.235Z"
  },
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "tokenType": "Bearer"
}
```

Рисунок 2.3 – Вход и получение *JWT*

Запрос к защищённому маршруту без заголовка *Authorization* отклоняется с кодом 401, с действительным токеном — выполняется. Ответы приведены на рисунках 2.4–2.5.

GET Send

REQUEST

Заголовки и тело не передаются

RESPONSE **Status: 401 Unauthorized**

```
{
  "error": {
    "code": "TOKEN_MISSING",
    "message": "Отсутствует токен доступа"
  }
}
```

Рисунок 2.4 – Запрос к защищённому маршруту без токена

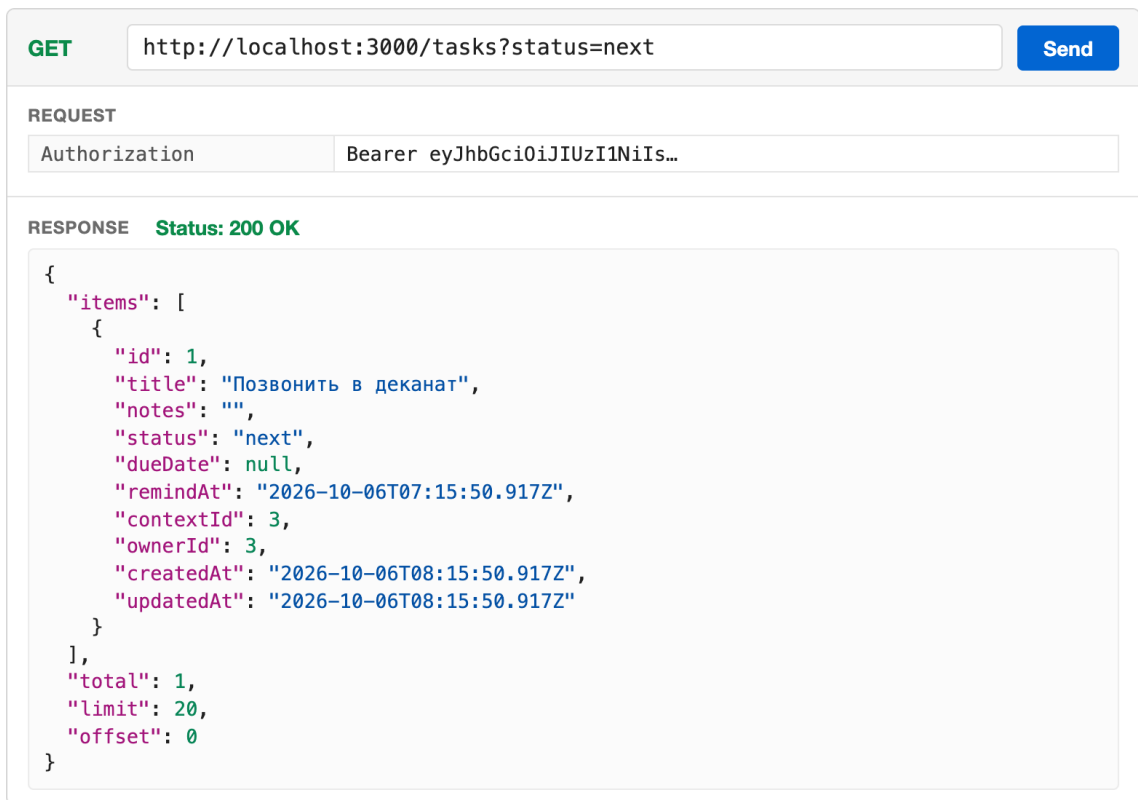


Рисунок 2.5 – Запрос к защищённому маршруту с токеном

Токен с изменённой подписью отклоняется с кодом *TOKEN_INVALID*, а событие *invalid_token* попадает в журнал безопасности. Ответ показан на рисунке 2.6.

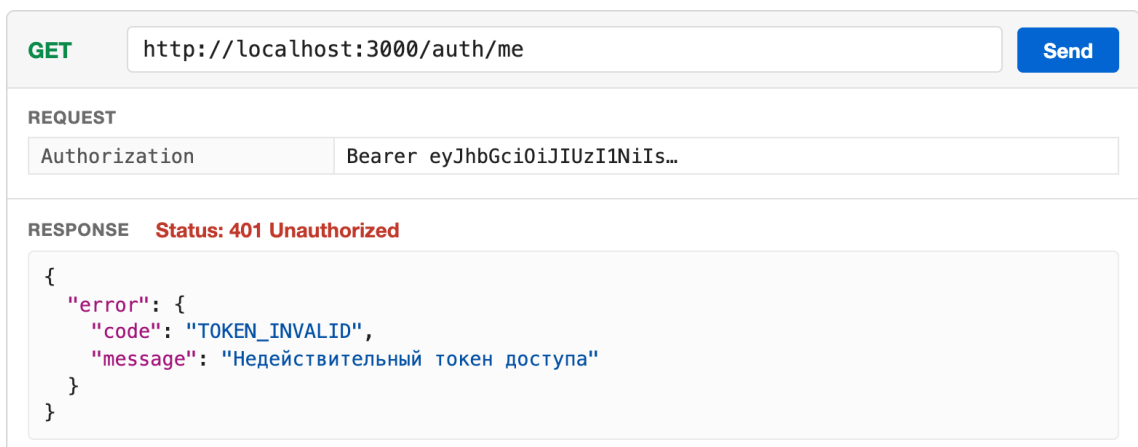


Рисунок 2.6 – Запрос с поддельным токеном

2.6 Ролевая модель

Пользователь с ролью *user* пытается удалить учётную запись администратора. *Middleware roleGuard* отклоняет запрос с кодом 403 ещё до проверки параметров, что показано на рисунке 2.7.

DELETE Send

REQUEST

Authorization	Bearer eyJhbGciOiJIUzI1NiIs...
---------------	--------------------------------

RESPONSE **Status: 403 Forbidden**

```
{
  "error": {
    "code": "INSUFFICIENT_ROLE",
    "message": "Операция доступна только для роли admin и выше"
  }
}
```

Рисунок 2.7 – Попытка административного действия от имени роли *user*

Модератору доступен просмотр пользователей, но назначение ролей остаётся за администратором. Отказ модератору приведён на рисунке 2.8.

PATCH Send

REQUEST

Authorization	Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type	application/json

```
{
  "role": "admin"
}
```

RESPONSE **Status: 403 Forbidden**

```
{
  "error": {
    "code": "INSUFFICIENT_ROLE",
    "message": "Операция доступна только для роли admin и выше"
  }
}
```

Рисунок 2.8 – Попытка модератора изменить роль пользователя

Пользователь не может прочитать чужую задачу даже при известном идентификаторе. Сервис отвечает кодом *FOREIGN_RESOURCE* и записывает событие *foreign_resource_access*, ответ представлен на рисунке 2.9.

GET

REQUEST

Authorization	Bearer eyJhbGciOiJIUzI1NiIs...
---------------	--------------------------------

RESPONSE **Status: 403 Forbidden**

```
{
  "error": {
    "code": "FOREIGN_RESOURCE",
    "message": "Нет доступа к чужому ресурсу"
  }
}
```

Рисунок 2.9 – Обращение к чужой задаче

2.7 Проверка входных данных

Схема *Joi* проверяет все поля сразу (*abortEarly: false*) и возвращает список нарушений с указанием источника и поля. Ответ на регистрацию с некорректным *email* и коротким паролем представлен на рисунке 2.10.

Даты задач ограничены диапазоном, который заведомо поддерживают и *JavaScript*, и *PostgreSQL*. Поэтому дата с годом -10000 не доходит до базы данных, а отклоняется с кодом 400:

```
const boundedDate = Joi.date().iso()
  .min('2000-01-01T00:00:00Z')
  .max('2100-12-31T23:59:59Z');
const isoDate = boundedDate.allow(null);
```

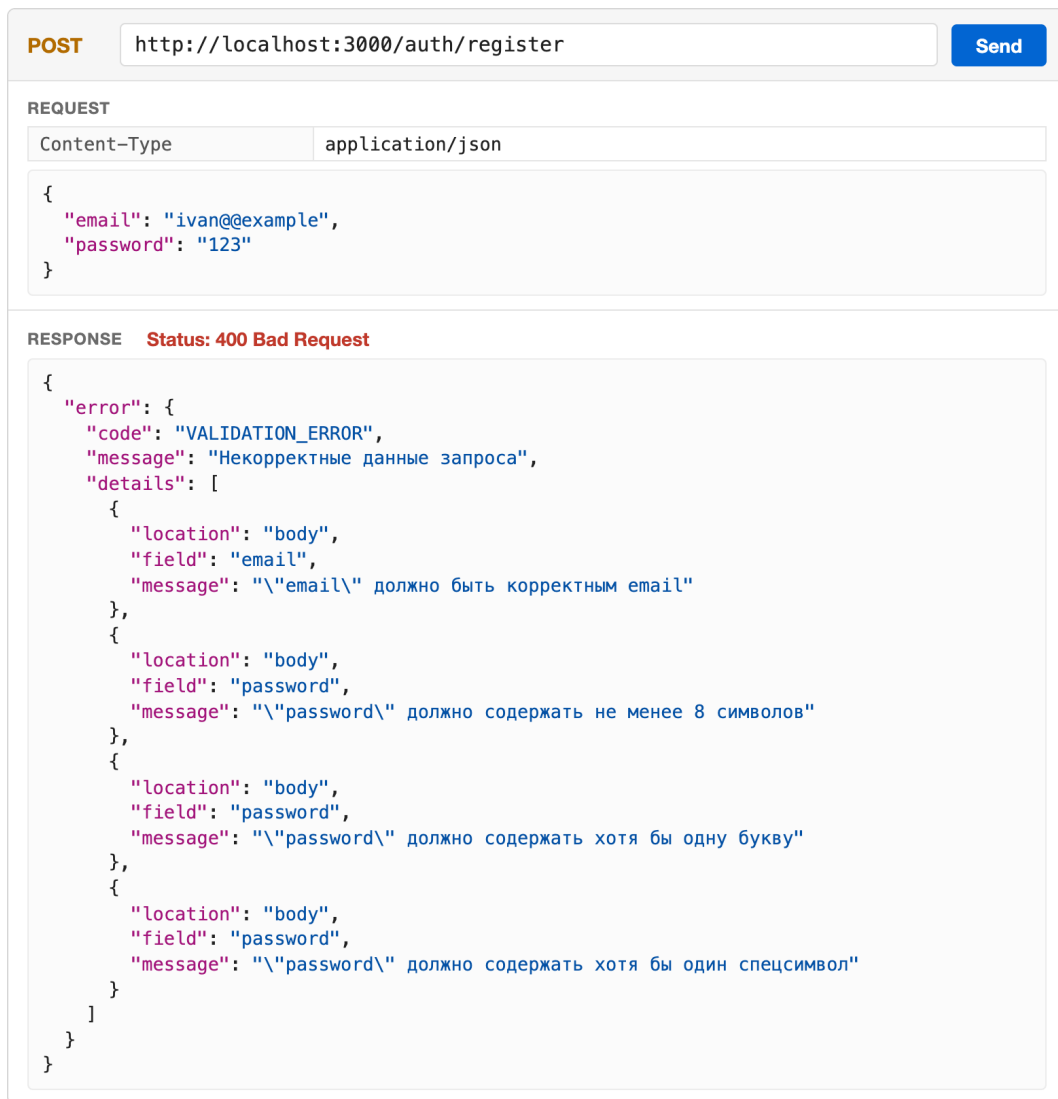


Рисунок 2.10 – Ошибки валидации при регистрации

2.8 Дополнительная мера 3: сложность пароля

Пароль должен содержать от 8 до 72 символов, хотя бы одну букву, цифру и спецсимвол. Верхняя граница 72 символа — предел *bcrypt*: всё, что длиннее, алгоритм молча отбрасывает. Правило описано в общей схеме:

```
const password = Joi.string()
  .min(8)
  .max(72)
  .pattern(/p{L}/u, 'хотя бы одну букву')
  .pattern(/d/, 'хотя бы одну цифру')
  .pattern(/[^p{L}\d\s]/u, 'хотя бы один спецсимвол');
```

Ответ на пароль из одних букв показан на рисунке 2.11.

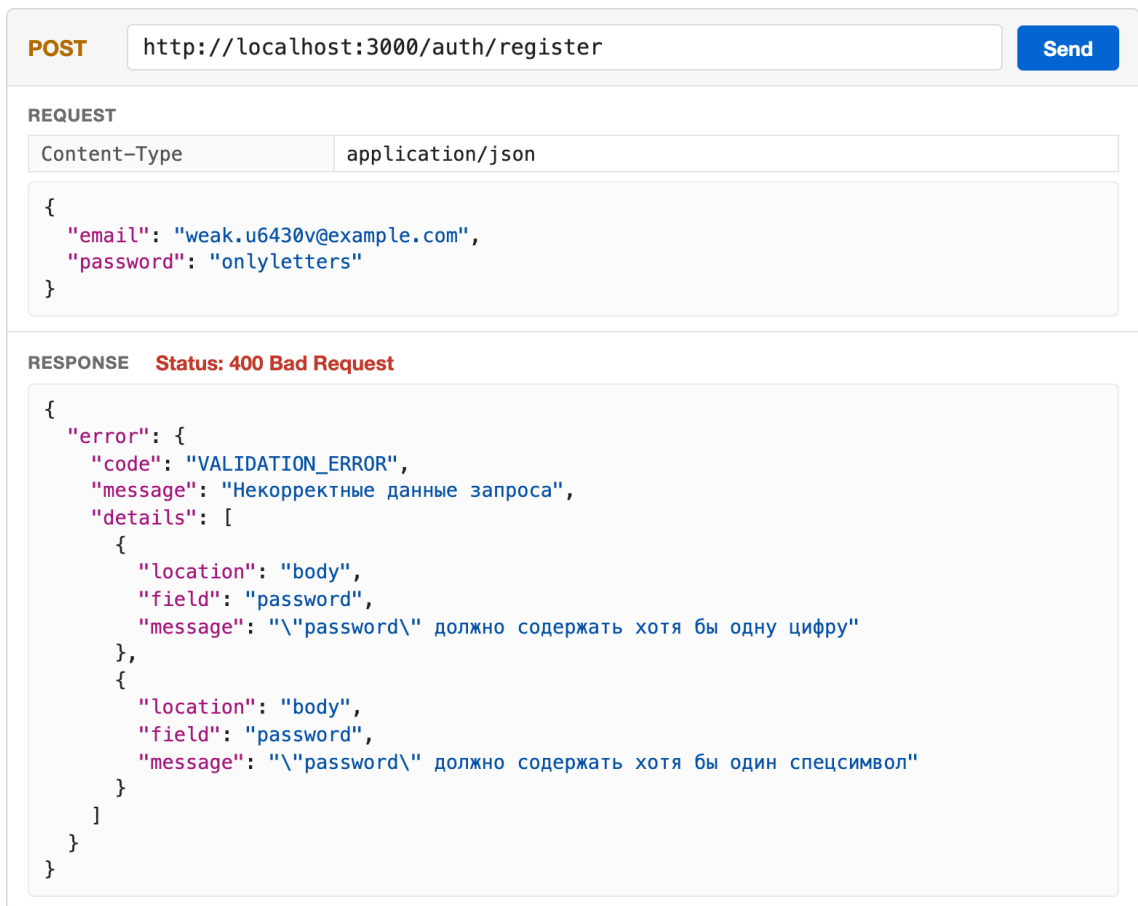


Рисунок 2.11 – Отказ в регистрации со слабым паролем

2.9 Дополнительная мера 5: *refresh*-токен в *httpOnly* cookie

Access-токен живёт 15 минут. *Refresh*-токен живёт 7 дней, хранится в *cookie*, недоступной *JavaScript*, а в базе данных лежит только его *SHA-256* хеш. Каждый вызов `/auth/refresh` отзывает предъявленный токен и выдаёт новую пару, что представлено на рисунке 2.12.

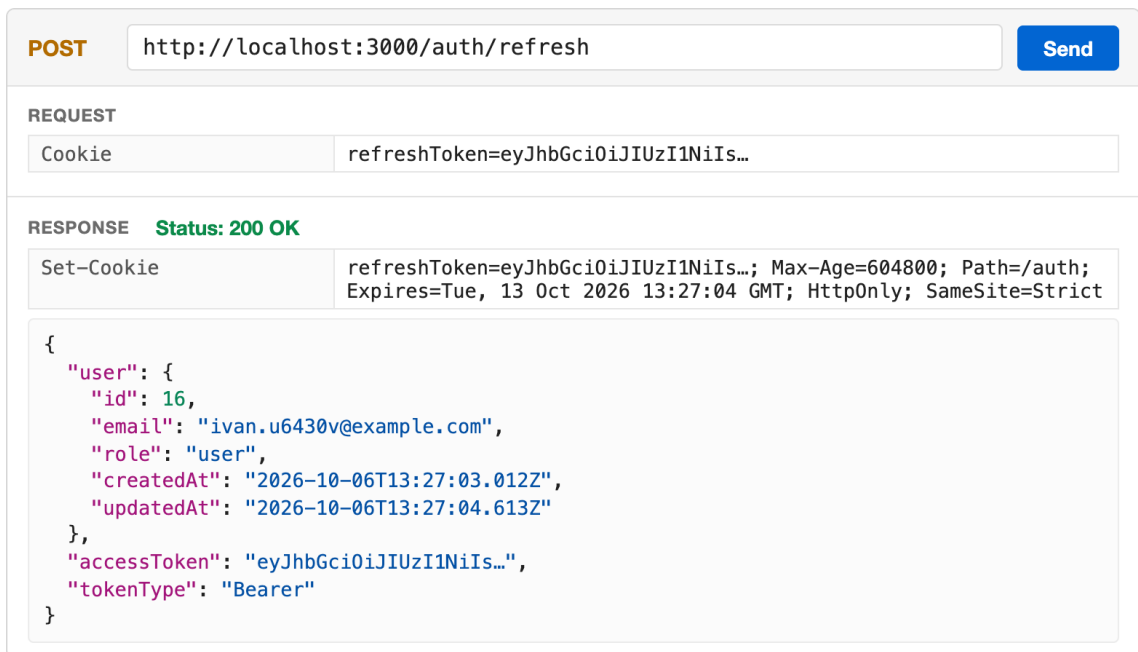


Рисунок 2.12 – Обновление пары токенов с ротацией

Если отозванный токен предъявляют повторно, значит, его копия оказалась у кого-то ещё. Сервис отклоняет запрос, отзывает все сессии пользователя и записывает событие *refresh_token_reuse*. Ответ приведён на рисунке 2.13.

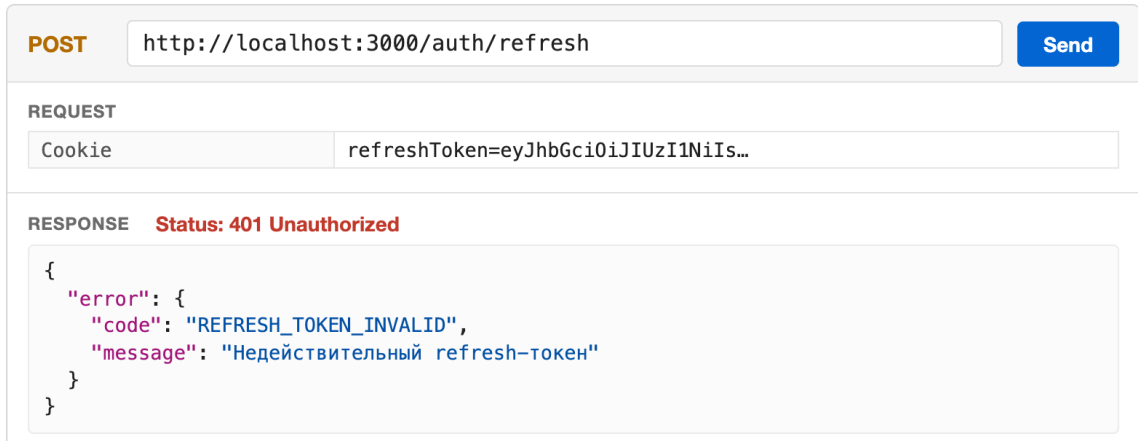


Рисунок 2.13 – Повторное предъявление отозванного *refresh*-токена

Ротация выполняется в транзакции с блокировкой строки, поэтому два одновременных запроса с одним токеном не получают две действующие пары:

```

const outcome = await sequelize.transaction(async (tx) => {
  const storedToken = await RefreshToken.findOne({
    where: { tokenHash }, transaction: tx,
    lock: tx.LOCK.UPDATE });
  if (!storedToken
    || String(storedToken.userId) !== payload.sub) {

```

```

    return { error: invalidRefreshToken() };
  }
  if (!storedToken.isActive()) {
    return { reusedToken: storedToken };
  }
  const user = await User.findByPk(storedToken.userId,
    { transaction: tx });
  const pair = await issueTokenPair(user, ip, tx);
  await storedToken.update({ revokedAt: new Date(),
    replacedByHash: hashToken(pair.refreshToken) },
    { transaction: tx });
  return { user, pair };
});

```

Выход отзывает *refresh*-токен и очищает *cookie*, после чего обновить сессию этим токеном нельзя. Ответы представлены на рисунках 2.14–2.15.

POST Send

REQUEST

Cookie	refreshToken=eyJhbGciOiJIUzI1NiIs...
--------	--------------------------------------

RESPONSE **Status: 204 No Content**

Set-Cookie	refreshToken=; Path=/auth; Expires=Thu, 01 Jan 1970 00:00:00 GMT; HttpOnly; SameSite=Strict
------------	---

Тело ответа пустое

Рисунок 2.14 – Выход из системы

POST Send

REQUEST

Cookie	refreshToken=eyJhbGciOiJIUzI1NiIs...
--------	--------------------------------------

RESPONSE **Status: 401 Unauthorized**

```

{
  "error": {
    "code": "REFRESH_TOKEN_INVALID",
    "message": "Недействительный refresh-токен"
  }
}

```

Рисунок 2.15 – Попытка обновить сессию после выхода

2.10 Дополнительная мера 6: блокировка после пяти неудачных входов

В таблице *users* хранятся счётчик *failedAttempts* и момент *lockUntil*. Пятая неудачная попытка закрывает вход на 15 минут. Пока блокировка действует, пароль не проверяется вовсе, поэтому даже верный пароль получает ответ 423, содержащий заголовок *Retry-After*. Проверка выполнена на свежезарегистрированной учётной записи: пять попыток с неверным паролем вернули 401, шестая с верным паролем — 423, как показано на рисунке 2.16.

POST Send

REQUEST

Content-Type	application/json
--------------	------------------

```
{
  "email": "victim.u6430v@example.com",
  "password": "....."
}
```

RESPONSE **Status: 423 Locked**

Retry-After	900
-------------	-----

```
{
  "error": {
    "code": "ACCOUNT_LOCKED",
    "message": "Учётная запись временно заблокирована после 5 неудачных попыток входа"
  }
}
```

Рисунок 2.16 – Вход в заблокированную учётную запись

Блокировка учётной записи защищает от подбора пароля к конкретному пользователю, а лимит запросов к */auth/** — от перебора с одного адреса по многим учётным записям. После 20 неуспешных запросов за 15 минут сервер отвечает 429, ответ приведён на рисунке 2.17.

POST Send

REQUEST

Content-Type	application/json
--------------	------------------

```
{
  "email": "bruteforce@example.com",
  "password": "Guess!8x"
}
```

RESPONSE **Status: 429 Too Many Requests**

RateLimit-Policy	20;w=900
RateLimit	limit=20, remaining=0, reset=856

```
{
  "error": {
    "code": "TOO_MANY_REQUESTS",
    "message": "Слишком много запросов, повторите позже"
  }
}
```

Рисунок 2.17 – Превышение лимита неуспешных запросов аутентификации

2.11 Дополнительная мера 9: защита от *mass assignment*

Схемы *Joi* не допускают неизвестных полей, поэтому поле *role* при регистрации и поле *ownerId* при создании задачи отклоняются с кодом 400. Ответы представлены на рисунках 2.18–2.19.

POST Send

REQUEST

Content-Type	application/json
--------------	------------------

```
{
  "email": "hacker.u6430v@example.com",
  "password": ".....",
  "role": "admin"
}
```

RESPONSE **Status: 400 Bad Request**

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Некорректные данные запроса",
    "details": [
      {
        "location": "body",
        "field": "role",
        "message": "Поле \"role\" не допускается"
      }
    ]
  }
}
```

Рисунок 2.18 – Попытка задать роль при регистрации

POST Send

REQUEST

Authorization	Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type	application/json

```
{
  "title": "Чужая задача",
  "ownerId": 1
}
```

RESPONSE **Status: 400 Bad Request**

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Некорректные данные запроса",
    "details": [
      {
        "location": "body",
        "field": "ownerId",
        "message": "Поле \"ownerId\" не допускается"
      }
    ]
  }
}
```

Рисунок 2.19 – Попытка создать задачу от имени другого владельца

Вторая линия защиты находится в сервисах: они копируют из входных данных только поля белого списка, а владельца берут из токена:

```
const WRITABLE_FIELDS = Object.freeze(['title', 'notes',
  'status', 'dueDate', 'remindAt', 'contextId']);

async function createTask(actor, input) {
  const attributes = { ...pickDefined(input, WRITABLE_FIELDS),
    ownerId: actor.id };
  ensureScheduledHasDueDate(attributes);
  await ensureContextOwnedBy(actor.id, attributes.contextId);
  return Task.create(attributes,
    { fields: [...WRITABLE_FIELDS, 'ownerId'] });
}
```

2.12 Дополнительные меры 12 и 15: размер и тип тела запроса

Тело запроса ограничено 10 КБ: заметки задачи не длиннее 2000 символов, и больший объём нужен только для попытки исчерпать память сервера. Ответ на тело размером около 12 КБ показан на рисунке 2.20.

The screenshot shows a REST client interface. At the top, a POST request is configured to `http://localhost:3000/tasks` with a 'Send' button. Below, the 'REQUEST' section shows headers: 'Authorization: Bearer eyJhbGciOiJIUzI1NiIs...' and 'Content-Type: application/json'. The request body is a JSON object: `{ "title": "Большая задача", "notes": "xxxxxxxxxx... (12 000 символов)" }`. The 'RESPONSE' section shows a status of 413: `Status: 413 Payload Too Large`. The response body is a JSON error object: `{ "error": { "code": "PAYLOAD_TOO_LARGE", "message": "Размер тела запроса превышает допустимый" } }`.

Рисунок 2.20 – Тело запроса больше 10 КБ

Тело запроса принимается только в формате *application/json*. Это закрывает отправку данных из *HTML*-формы с другого сайта, которая возможна лишь с типами *text/plain*, *application/x-www-form-urlencoded* и *multipart/form-data*. Ответ на тело *text/plain* приведён на рисунке 2.21.

POST Send

REQUEST

Authorization	Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type	text/plain

title=Купить молоко

RESPONSE **Status: 415 Unsupported Media Type**

```
{
  "error": {
    "code": "UNSUPPORTED_MEDIA_TYPE",
    "message": "Тело запроса должно иметь Content-Type: application/json"
  }
}
```

Рисунок 2.21 – Тело запроса в формате *text/plain*

2.13 Заголовки безопасности

Helmet добавляет к каждому ответу *API* строгую политику *CSP default-src 'none'*, запрет встраивания во фрейм, *HSTS* и запрет угадывания типа содержимого. *API* отдаёт только *JSON*, поэтому политика не разрешает загрузку никаких ресурсов. Заголовки ответа представлены на рисунке 2.22.

GET Send

REQUEST

Заголовки и тело не передаются

RESPONSE **Status: 200 OK**

Content-Security-Policy	default-src 'none';frame-ancestors 'none';base-uri 'none';form-action 'none'
X-Frame-Options	DENY
Strict-Transport-Security	max-age=31536000; includeSubDomains
X-Content-Type-Options	nosniff
Cross-Origin-Resource-Policy	same-site

```
{
  "status": "ok"
}
```

Рисунок 2.22 – Заголовки безопасности в ответе *API*

2.14 Дополнительная мера 19: санитизация *HTML*

Все строки тела запроса, кроме пароля, проходят через *sanitize-html* без разрешённых тегов. Тег *script* удаляется вместе с содержимым, остальные теги вырезаются, текст сохраняется. Результат представлен на рисунке 2.23.

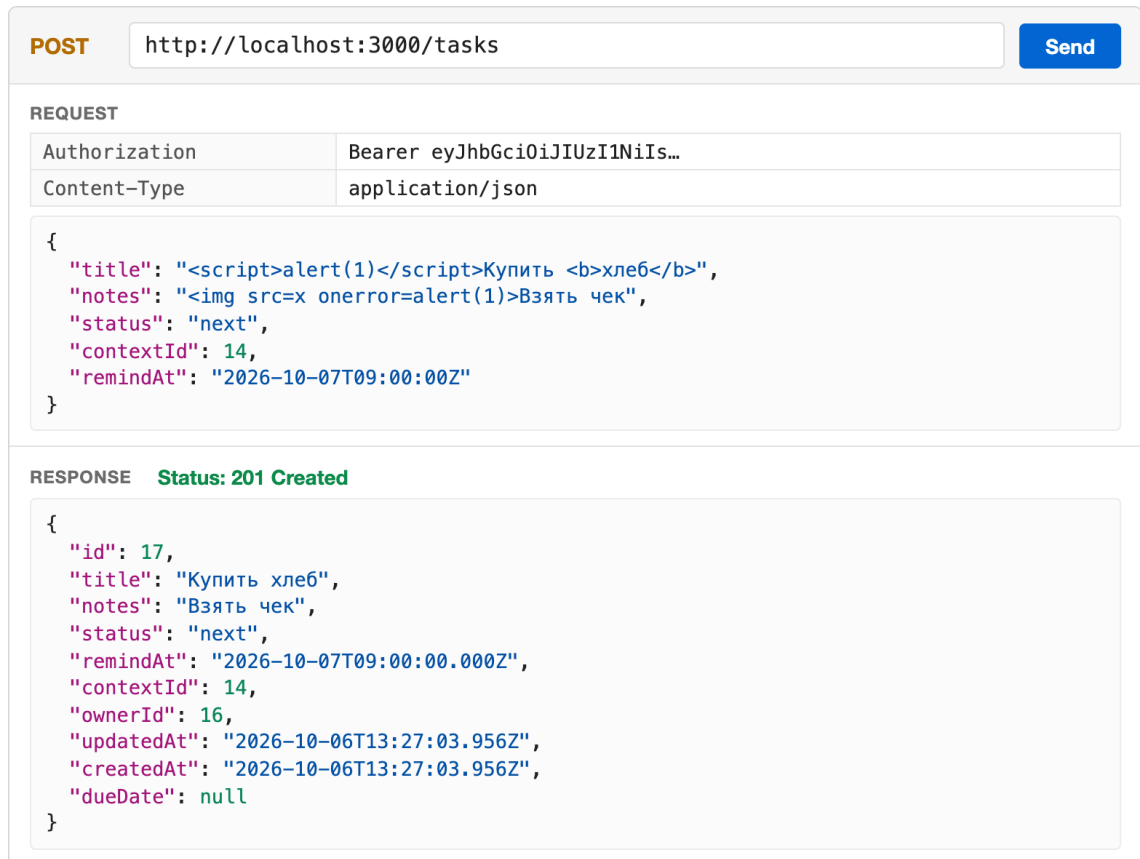


Рисунок 2.23 – Санитизация *HTML* в полях задачи

2.15 Дополнительная мера 22: *CORS* по белому списку

Разрешённые источники перечислены в переменной окружения *CORS_ORIGINS*. Запрос с чужого *Origin* получает 403 и записывается в журнал, запрос с разрешённого источника получает заголовки *Access-Control-Allow-Origin* и *Access-Control-Allow-Credentials*. Ответы приведены на рисунках 2.24–2.25.

GET Send

REQUEST

Origin	https://evil.example
--------	----------------------

RESPONSE **Status: 403 Forbidden**

```
{
  "error": {
    "code": "CORS_REJECTED",
    "message": "Источник запроса не разрешён политикой CORS"
  }
}
```

Рисунок 2.24 – Запрос с неразрешённого источника

GET Send

REQUEST

Origin	http://localhost:5173
--------	-----------------------

RESPONSE **Status: 200 OK**

Access-Control-Allow-Origin	http://localhost:5173
Access-Control-Allow-Credentials	true
Vary	Origin

```
{
  "items": [
    {
      "id": 3,
      "name": "@calls",
      "createdAt": "2026-10-06T08:15:50.911Z"
    },
    {
      "id": 2,
      "name": "@work",
      "createdAt": "2026-10-06T08:15:50.907Z"
    }
  ],
  "total": 2,
  "limit": 2,
  "offset": 0
}
```

Рисунок 2.25 – Запрос с разрешённого источника

2.16 Дополнительная мера 24: ошибки без деталей

Ответ на ошибку входа не раскрывает, существует ли учётная запись: неизвестный *email* и неверный пароль дают одинаковый код и текст. Для неизвестного *email* сервер всё равно выполняет сравнение *bcrypt* с фиктивным хешем, чтобы время ответа тоже не отличалось. Ответы показаны на рисунках 2.26–2.27.

POST

REQUEST

Content-Type	application/json
--------------	------------------

```
{
  "email": "nobody.u6430v@example.com",
  "password": "Wrong!pass1"
}
```

RESPONSE **Status: 401 Unauthorized**

```
{
  "error": {
    "code": "INVALID_CREDENTIALS",
    "message": "Неверный email или пароль"
  }
}
```

Рисунок 2.26 – Вход с несуществующим *email*

POST

REQUEST

Content-Type	application/json
--------------	------------------

```
{
  "email": "user@gtd.local",
  "password": "Wrong!pass1"
}
```

RESPONSE **Status: 401 Unauthorized**

```
{
  "error": {
    "code": "INVALID_CREDENTIALS",
    "message": "Неверный email или пароль"
  }
}
```

Рисунок 2.27 – Вход с неверным паролем существующего пользователя

Непредвиденная ошибка в режиме *development* возвращает блок *debug* с именем, сообщением и стеком, а в режиме *production* — только общий текст. Для демонстрации рядом с основным сервером запущен второй экземпляр с *NODE_ENV=production* на порту 3001, после чего контейнер *PostgreSQL* остановлен и к обоим экземплярам отправлен запрос, которому нужна база данных. Затем база запущена снова. Ответы приведены на рисунках 2.28–2.29.

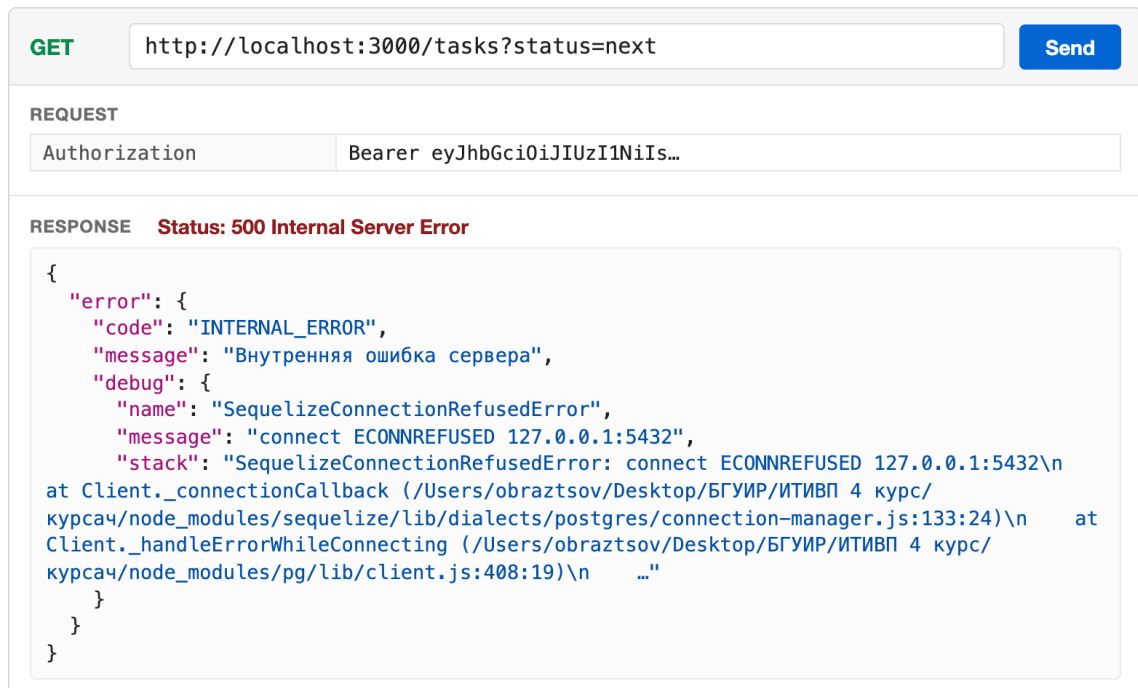


Рисунок 2.28 – Внутренняя ошибка в режиме *development*

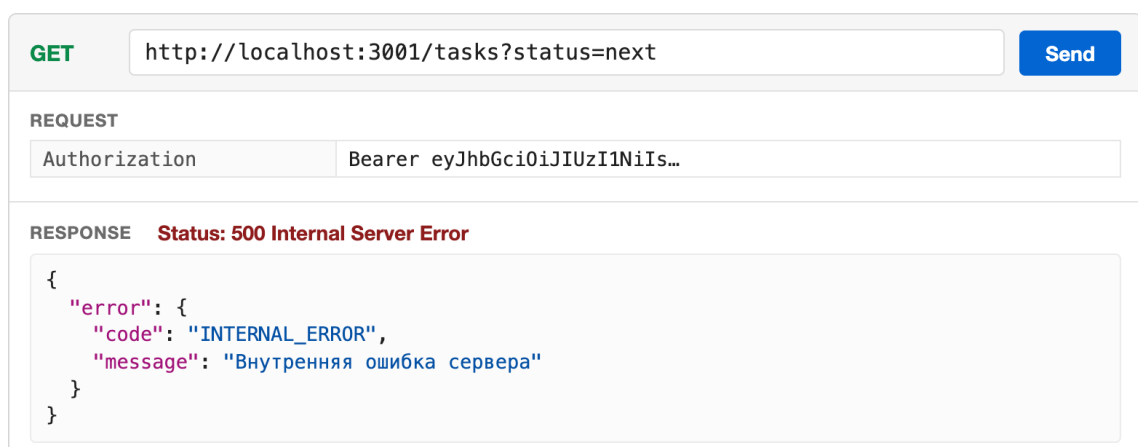


Рисунок 2.29 – Внутренняя ошибка в режиме *production*

Обработчик ошибок переводит известные ошибки в ответ с кодом, а все прочие записывает в журнал и заменяет общим сообщением:

```

function errorHandler(error, req, res, next) {
  const knownError = toAppError(error);
  if (knownError) {
    if (knownError.status >= 500) {
      logger.error(knownError.message,
        { stack: knownError.stack, path: req.originalUrl });
    }
    res.set(knownError.headers || {});
    res.status(knownError.status)
      .json(buildBody(knownError));
    return;
  }
  logger.error(error.message, { name: error.name,
    stack: error.stack, method: req.method,
    path: req.originalUrl });
  const serverError = new AppError(GENERIC_SERVER_MESSAGE);
  res.status(serverError.status)
    .json(buildBody(serverError, error));
}

function buildBody(appError, originalError) {
  const body = { code: appError.code,
    message: appError.message };
  if (appError.details) {
    body.details = appError.details;
  }
  if (!config.isProduction && originalError) {
    body.debug = { name: originalError.name,
      message: originalError.message,
      stack: originalError.stack };
  }
  return { error: body };
}

```

2.17 Дополнительная мера 30: журнал ошибок с ротацией

Ошибки уровня *error* пишутся в файл *logs/error-ГГГГ-ММ-ДД.log*. Файлы ротируются ежедневно и при достижении 10 МБ, старые архивируются *gzip* и хранятся 14 дней. Модуль ротации ведёт служебный файл *audit.json* со списком файлов и сроком хранения. На следующий день после работы с сервером файл за прошлые сутки уже заархивирован. Каталог журналов и запись об ошибке из предыдущего подраздела представлены на рисунке 2.30.

```
logs — журнал ошибок с ротацией

$ ls -la logs/
787  .3b5e383e3d0afac694edafe66ec180545ddad80c-audit.json
783  .666d93052a6c8c551731049210fa33d7e3f3af7a-audit.json
793  .b14b4463af5f01c32531f8347bedae798eaeed9d-audit.json
9779 app-2026-10-06.log.gz
27161 app-2026-10-07.log
1000 error-2026-10-06.log.gz
2366 error-2026-10-07.log
74107 security-2026-10-06.log
13384 security-2026-10-07.log

$ cat logs/.3b5e...-audit.json | jq '.keep, .files[].name'
{"days":true,"amount":14}
"logs/error-2026-10-06.log"
"logs/error-2026-10-07.log"

$ tail -n 1 logs/error-2026-10-07.log
{"level":"error","message":"connect ECONNREFUSED
127.0.0.1:5432","method":"GET","name":"SequelizeConnectionRefusedError","path":"/tasks?
status=next","stack":"SequelizeConnectionRefusedError: connect ECONNREFUSED 127.0.0.1:5432\n    at
Client._connectionCallback (.../sequelize/lib/dialects/postgres/connection-manager.js) ...","timestamp":"2026-
10-07T10:01:35.251Z"}
```

Рисунок 2.30 – Журнал ошибок с ротацией

2.18 Журнал подозрительных действий

Отдельный журнал *logs/security-ГГГГ-ММ-ДД.log* содержит по одной *JSON*-строке на событие с *IP*-адресом, методом, путём, *user-agent* и идентификатором пользователя. Записи, появившиеся при выполнении описанных выше запросов, приведены на рисунке 2.31.

```
logs — security-2026-10-06.log

$ jq -c 'del(.channel, .message)' logs/security-2026-10-06.log
{"event":"invalid_token","ip":"","1","level":"warn","method":"GET","path":"/auth/me","reason":"invalid
signature","timestamp":"2026-10-06T13:27:03.478Z","userAgent":"node"}
{"event":"insufficient_role","ip":"","1","level":"warn","method":"DELETE","path":"/users/1","requiredRole":
"admin","role":"user","timestamp":"2026-10-06T13:27:03.480Z","userAgent":"node","userId":3}
{"action":"read","event":"foreign_resource_access","ip":"","1","level":"warn","method":"GET","ownerId":3,"p
ath":"/tasks/3","resourceId":3,"resourceType":"Task","role":"user","timestamp":"2026-10-
06T13:27:03.938Z","userAgent":"node","userId":16}
{"event":"mass_assignment_attempt","fields":
[{"role"},"ip":"","1","level":"warn","method":"POST","path":"/auth/register","timestamp":"2026-10-
06T13:27:03.941Z","userAgent":"node"}
{"event":"cors_origin_rejected","ip":"","1","level":"warn","method":"GET","origin":"https://evil.example","
path":"/contexts/public","timestamp":"2026-10-06T13:27:03.959Z","userAgent":"node"}
{"email":"nobody.u6430v@example.com","event":"login_failed","ip":"","1","level":"warn","timestamp":"2026-
10-06T13:27:04.177Z","userAgent":"node","userExists":false}
{"event":"refresh_token_reuse","ip":"","1","level":"warn","timestamp":"2026-10-
06T13:27:04.633Z","userId":16}
{"event":"account_locked","ip":"","1","level":"warn","lockUntil":"2026-10-
06T13:42:06.178Z","timestamp":"2026-10-06T13:27:06.180Z","userAgent":"node","userId":17}
{"event":"login_blocked_locked","ip":"","1","level":"warn","timestamp":"2026-10-
06T13:27:06.182Z","userAgent":"node","userId":17}
{"event":"rate_limit_exceeded","ip":"","1","level":"warn","limit":20,"limiter":"auth","method":"POST","path
":"/auth/login","timestamp":"2026-10-06T13:27:46.800Z","userAgent":"node"}
```

Рисунок 2.31 – Журнал подозрительных действий

2.19 Документация и автоматическое тестирование

Спецификация *OpenAPI* 3.0 хранится в файле *docs/openapi.json* и отображается через *Swagger UI* по адресу */api-docs*. Для страницы документации задана отдельная политика *CSP*, разрешающая только собственные скрипты и стили. Страница документации и описание маршрута входа представлены на рисунках 2.32–2.33.

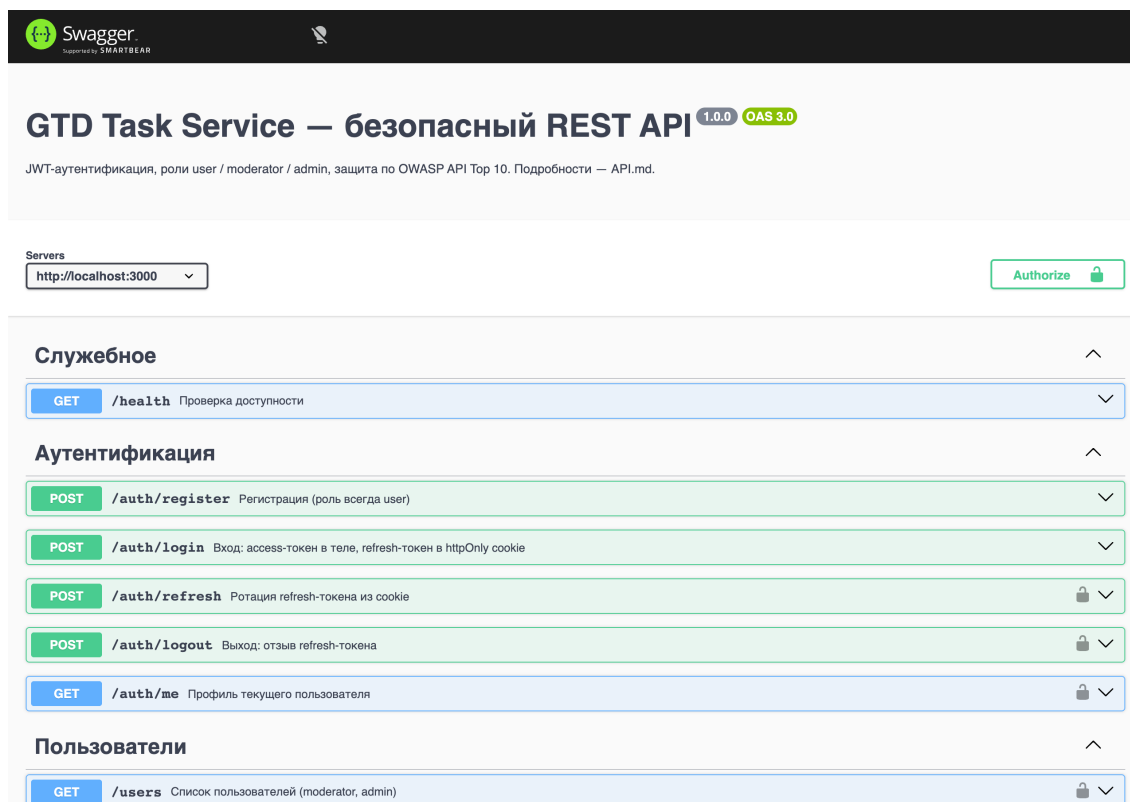


Рисунок 2.32 – *Swagger UI* сервиса

POST

/auth/login

Вход: access-токен в теле, refresh-токен в httpOnly cookie

Try it out

Parameters

No parameters

Request body required

application/json

Example Value

Schema

```
{
  "email": "ivan@example.com",
  "password": "Str0ng!pass"
}
```

Responses

Code	Description	Links
200	Успешный вход	No links
	Media type application/json Controls Accept header. Example Value Schema <pre>{ "user": { "id": 0, "email": "user@example.com", "role": "user", "createdAt": "2026-10-07T10:10:16.159Z", "updatedAt": "2026-10-07T10:10:16.159Z" }, "accessToken": "string", "tokenType": "Bearer" }</pre>	
400	Ошибка валидации или некорректный JSON. Коды: VALIDATION_ERROR	No links
	Media type application/json Example Value Schema <pre>{ "error": { "code": "string", "message": "string", "details": [{ "location": "string", "field": "string", "message": "string" }] } }</pre>	
401	Нет или недействительный токен, неверные учётные данные. Коды: TOKEN_MISSING, TOKEN_INVALID, TOKEN_EXPIRED, INVALID_CREDENTIALS, REFRESH_TOKEN_MISSING, REFRESH_TOKEN_INVALID	No links
	Media type application/json Example Value Schema <pre>{ "error": { "code": "string", "message": "string", "details": [{ "location": "string", "field": "string", "message": "string" }] } }</pre>	
423	Учётная запись заблокирована на 15 минут после 5 неудачных входов, заголовок Retry-After. Коды: ACCOUNT_LOCKED	No links
	Media type application/json Example Value Schema <pre>{ "error": { "code": "string", "message": "string", "details": [{ "location": "string", "field": "string", "message": "string" }] } }</pre>	
429	Превышен лимит запросов. Коды: TOO_MANY_REQUESTS	No links
	Media type application/json Example Value Schema <pre>{ "error": { "code": "string", "message": "string", "details": [{ "location": "string", "field": "string", "message": "string" }] } }</pre>	

Рисунок 2.33 – Описание маршрута *POST /auth/login* в *Swagger UI*

Коллекция *Postman* содержит 102 запроса в восьми папках с 184 проверками, включая коды 401, 403, 404, 409, 413, 415, 423 и 429. Коллекция запущена через *Newman* на сервере с чистыми счётчиками лимитов: выполнено 123 *HTTP*-запроса (часть проверок отправляет дополнительные запросы из скриптов), 184 проверки, ошибок нет. Фрагмент прогона и итоговая таблица приведены на рисунке 2.34.

```

Newman — прогон коллекции

$ npx newman run postman/gtd-secure-api.postman_collection.json --env-var baseUrl=http://localhost:3000
...
Folder 05. Блокировка после 5 неудачных входов
Sub-folder Регистрация жертвы
  POST http://localhost:3000/auth/register [201 Created, 924B, 222ms]
  Pass Статус 201

Sub-folder Неверный пароль #1 → 401
  POST http://localhost:3000/auth/login [401 Unauthorized, 871B, 218ms]
  Pass Статус 401
  Pass Код ошибки INVALID_CREDENTIALS

Sub-folder Неверный пароль #2 → 401
  POST http://localhost:3000/auth/login [401 Unauthorized, 871B, 216ms]
  Pass Статус 401
  Pass Код ошибки INVALID_CREDENTIALS

Sub-folder Неверный пароль #3 → 401
  POST http://localhost:3000/auth/login [401 Unauthorized, 870B, 218ms]
  Pass Статус 401
  Pass Код ошибки INVALID_CREDENTIALS

Sub-folder Неверный пароль #4 → 401
  POST http://localhost:3000/auth/login [401 Unauthorized, 870B, 216ms]
  Pass Статус 401
  Pass Код ошибки INVALID_CREDENTIALS

Sub-folder Неверный пароль #5 → 401
  POST http://localhost:3000/auth/login [401 Unauthorized, 870B, 218ms]
  Pass Статус 401
  Pass Код ошибки INVALID_CREDENTIALS

Sub-folder Верный пароль во время блокировки → 423
  POST http://localhost:3000/auth/login [423 Locked, 965B, 3ms]
  Pass Статус 423
...

=====
|                               | executed | failed |
|-----|-----|-----|
| iterations |         1 |      0 |
|-----|-----|-----|
| requests   |        123 |      0 |
|-----|-----|-----|
| test-scripts |       102 |      0 |
|-----|-----|-----|
| prerequisite-scripts |         4 |      0 |
|-----|-----|-----|
| assertions  |        184 |      0 |
|-----|-----|-----|
| total run duration: 4.8s |
|-----|-----|-----|

```

Рисунок 2.34 – Прогон коллекции *Postman* через *Newman*

3 ОБОСНОВАНИЕ ВЫБРАННЫХ ДОПОЛНИТЕЛЬНЫХ МЕР

Сервис хранит личные задачи и напоминания, поэтому атакующему нужен доступ к данным конкретного человека. Меры выбраны по трём направлениям: защита учётной записи, сессии и чужих данных. Обоснование приведено в таблице 3.1.

Таблица 3.1 – Дополнительные меры защиты и их обоснование

Номер	Мера	Обоснование для <i>GTD</i> -сервиса
3	Сложность пароля	Учётная запись открывает весь план жизни пользователя, поэтому пароль из словаря недопустим
5	Короткий <i>access</i> -токен, <i>refresh</i> в <i>httpOnly</i> <i>cookie</i> с ротацией	Клиенты держат сессию днями. Короткий <i>access</i> -токен ограничивает ущерб от утечки, <i>cookie</i> недоступна <i>XSS</i> , ротация выявляет кражу
6	Блокировка после 5 неудачных входов	Перебор пароля к известному <i>email</i> сводится к пяти попыткам за 15 минут
9	Защита от <i>mass assignment</i>	Поля <i>role</i> и <i>ownerId</i> дают повышение прав и подмену владельца задачи — прямой путь к чужим данным
12	Лимит тела 10 КБ	Самое длинное поле — заметки до 2000 символов; большие тела нужны только для атаки на память
15	Только <i>application/json</i>	Закрывает отправку запросов <i>HTML</i> -формой с другого сайта и путаницу при разборе тела
19	Санитизация <i>HTML</i>	Заголовки и заметки задач выводятся в веб-клиенте и в уведомлениях, внедрённый скрипт выполнился бы у владельца
22	<i>CORS</i> по белому списку	Браузер пользователя не позволит чужому сайту читать ответы <i>API</i> с его <i>cookie</i>
24	Ошибки без деталей	Стек раскрывает устройство приложения, одинаковый ответ на вход скрывает существование <i>email</i>
30	Журнал ошибок с ротацией	Ошибки сохраняются для разбора, а журнал не заполняет диск

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1 Реализованы роли *user*, *moderator* и *admin* с рангами 1, 2 и 3. Пользователь работает только со своими задачами и контекстами и может опубликовать свой контекст в каталоге. Модератор дополнительно читает чужие задачи, контексты и список пользователей — это нужно для разбора жалоб и поддержки. Администратор дополнительно удаляет чужие данные, назначает роли и удаляет учётные записи. Изменять чужие задачи не может никто, поскольку задача — личные данные владельца. Публичный каталог контекстов доступен без входа. Схема повторяет модель курсового проекта: личный планировщик с общим каталогом контекстов и модерацией.

2 Пароли хранятся в виде хеша *bcrypt* с 12 раундами, соль *bcrypt* генерирует сам и хранит внутри хеша. От перебора защищают блокировка учётной записи после пяти неудачных входов на 15 минут, лимит 20 неуспешных запросов к */auth/** за 15 минут с одного *IP* и требования к сложности пароля. При неизвестном *email* выполняется сравнение с фиктивным хешем, поэтому время ответа не выдаёт существование учётной записи.

3 В коде нет сырого *SQL*: все запросы строятся методами *Sequelize* (*findByPk*, *findAndCountAll*, *create*, *update*), которые передают значения в *PostgreSQL* как параметры, а не подставляют их в текст запроса. Дополнительно *Joi* приводит идентификаторы к целым числам, а парсер строки запроса в режиме *simple* не создаёт вложенных объектов, через которые в *ORM* можно было бы передать операторы.

4 Используется библиотека *Joi*. Схема описывается для каждого источника данных каждого маршрута, неизвестные поля запрещены. Проверка *email* выглядит так: *Joi.string().trim().lowercase().email({ tlds: { allow: false } }).max(254)*. Строка обрезается от пробелов, приводится к нижнему регистру, проверяется на формат адреса и длину 254 символа — предел стандарта и размер столбца в базе.

5 От *XSS* защищают три механизма. На входе *sanitize-html* удаляет из строк все *HTML*-теги. На выходе *Express* экранирует символы *<*, *>* и *&* в *JSON* как *\u003c*, *\u003e* и *\u0026*. *Helmet* задаёт заголовки *Content-Security-Policy: default-src 'none'*, *X-Content-Type-Options: nosniff* и *X-Frame-Options: DENY*, поэтому браузер не исполнит ответ *API* как страницу и не встроит его во фрейм. Заголовки показаны на рисунке 2.22.

6 Глобальный лимит реализован библиотекой *express-rate-limit*: 300 запросов за 15 минут с одного *IP* для всех маршрутов. Для маршрутов аутентификации действует отдельный лимит 20 неуспешных запросов. Ответ сверх лимита — 429, содержащий заголовки *RateLimit* и *RateLimit-Policy*, событие

rate_limit_exceeded пишется в журнал. Счётчики хранятся в памяти процесса; для нескольких экземпляров сервера нужен общий *store*, например *Redis*.

7 *JWT* — токен из трёх частей в *Base64URL*: заголовок с алгоритмом, полезная нагрузка с утверждениями и подпись. Сервер проверяет подпись секретом и не хранит сессию. *Access*-токен подписывается *HS256*, содержит *sub* (*id* пользователя), *role*, *type*, *iss*, *aud* и *exp* и живёт 15 минут. Клиент хранит его в памяти и передаёт в заголовке *Authorization: Bearer*. *Refresh*-токен подписывается другим секретом, передаётся только в *cookie HttpOnly; SameSite=Strict; Path=/auth*, а в базе хранится его *SHA-256* хеш.

8 Выбраны меры 3, 5, 6, 9, 12, 15, 19, 22, 24 и 30. Причины приведены в таблице 3.1: сервис хранит личные данные, поэтому главное — защитить учётную запись от подбора, сессию от кражи и записи от изменения чужими руками.

9 Подозрительные действия пишутся отдельным логгером *winston* в файл *logs/security-ГТТГ-ММ-ДД.log* и в консоль. Фиксируются неудачный вход, блокировка, вход в заблокированную учётную запись, поддельный токен, повторное использование *refresh*-токена, отказ по роли, обращение к чужому ресурсу, превышение лимита, запрос с чужого *Origin* и лишние поля в теле. Пример реальной записи из журнала (в файле это одна строка):

```
{ "action": "read", "channel": "security",  
  "event": "foreign_resource_access", "ip": "::1",  
  "level": "warn", "message": "foreign_resource_access",  
  "method": "GET", "ownerId": 3, "path": "/tasks/3",  
  "resourceId": 3, "resourceType": "Task", "role": "user",  
  "timestamp": "2026-10-06T13:27:03.938Z",  
  "userAgent": "node", "userId": 16 }
```

Запись показывает, что пользователь с *id* 16 и ролью *user* пытался прочитать задачу 3, принадлежащую пользователю 3.

10 *Access*-токен передаётся в заголовке *Authorization*, который браузер не добавляет к запросу сам, поэтому *CSRF* для основных маршрутов невозможен. *Refresh-cookie* имеет *SameSite=Strict* и путь */auth*, то есть не отправляется с запросами, инициированными другим сайтом. Кроме того, тело принимается только как *application/json*, а *HTML*-форма другого сайта такой тип отправить не может, и *CORS* пропускает только источники из белого списка.

11 Используются *refresh*-токены с ротацией. При входе выдаётся пара токенов, при обновлении старый *refresh*-токен отзывается и заменяется новым, связь фиксируется в поле *replacedByHash*. Повторное предъявление отозванного токена считается кражей: отзываются все сессии пользователя. Выход отзывает токен и очищает *cookie*.

12 В *production* на непредвиденную ошибку возвращается только «Внутренняя ошибка сервера» с кодом *INTERNAL_ERROR*, подробности уходят в журнал ошибок. Ошибка входа одинакова для неизвестного *email* и неверного пароля. Сообщения об ошибках валидации называют поле и правило, но не раскрывают устройство приложения.

13 Размер тела ограничен 10 КБ параметром *limit* в *express.json*. Превышение даёт ответ 413 и код *PAYLOAD_TOO_LARGE* до того, как тело будет разобрано целиком.

14 Пароль должен содержать от 8 до 72 символов, хотя бы одну букву, одну цифру и один спецсимвол. Правило применяется при регистрации и при создании учётных записей *seed*.

15 *API* тестировалось коллекцией *Postman*, запущенной через *Newman*: 102 запроса, 184 проверки, 0 ошибок. Негативные сценарии: запрос без токена и с поддельным токеном, токен удалённого пользователя, действия *user* и *moderator* сверх своих прав, чтение и изменение чужих задач и контекстов, некорректные *email* и пароль, лишние поля *role* и *ownerId*, неизвестные *query*-параметры, неверный *JSON*, тело больше 10 КБ, тип *text/plain*, *HTML* и скрипты в полях, чужой *Origin*, повторный *refresh*, *refresh* после выхода, пять неудачных входов подряд и перебор пароля до 429. Дополнительно запросы выполнены вручную, их результаты приведены в разделе 2.

16 Покрытие категорий *OWASP Top 10:2021* приведено в таблице 4.1.

Таблица 4.1 – Покрытие *OWASP Top 10:2021*

Категория	Реализованные меры
<i>A01 Broken Access Control</i>	<i>roleGuard</i> , проверка владельца в <i>accessPolicy</i> , запрет чужого <i>ownerId</i> в фильтрах, защита от <i>mass assignment</i> , <i>CORS</i> по белому списку
<i>A02 Cryptographic Failures</i>	<i>bcrypt</i> с 12 раундами, секреты <i>JWT</i> не короче 32 символов и различные для двух типов токенов, <i>SHA-256</i> хеш <i>refresh</i> -токена в базе, <i>HSTS</i> , флаг <i>Secure</i> у <i>cookie</i> в <i>production</i>
<i>A03 Injection</i>	параметризованные запросы <i>Sequelize</i> , <i>Joi</i> для всех входных данных, санитизация <i>HTML</i> , экранирование <i>JSON</i> , <i>CSP</i>
<i>A04 Insecure Design</i>	блокировка учётной записи, ротация и обнаружение повторного <i>refresh</i> -токена, запрет изменения чужих задач для всех ролей, роль из базы данных при каждом запросе

Продолжение таблицы 4.1

Категория	Реализованные меры
<i>A05 Security Misconfiguration</i>	<i>helmet</i> , отключён <i>X-Powered-By</i> , ошибки без деталей в <i>production</i> , только <i>application/json</i> , <i>PostgreSQL</i> слушает только 127.0.0.1, секреты в <i>.env</i> вне репозитория
<i>A06 Vulnerable and Outdated Components</i>	актуальные версии зависимостей, зафиксированные в <i>package-lock.json</i> . Аудит <i>npm</i> показывает две уязвимости средней степени в пакете <i>uuid</i> , подключаемом через <i>Sequelize</i> ; уязвимые функции <i>v3</i> , <i>v5</i> , <i>v6</i> с буфером в проекте не вызываются
<i>A07 Identification and Authentication Failures</i>	сложность пароля, блокировка, лимит запросов к <i>/auth/*</i> , <i>access</i> -токен на 15 минут, одинаковый ответ и время ответа при неверном <i>email</i> и пароле
<i>A08 Software and Data Integrity Failures</i>	проверка подписи <i>JWT</i> с явным списком алгоритмов, издателем, аудиторией и типом токена; <i>lock</i> -файл зависимостей
<i>A09 Security Logging and Monitoring Failures</i>	журнал подозрительных действий, журнал ошибок с ротацией, журнал <i>HTTP</i> -запросов
<i>A10 Server-Side Request Forgery</i>	сервис не выполняет запросы по адресам, полученным от клиента, поэтому вектор отсутствует

ЗАКЛЮЧЕНИЕ

В ходе работы разработан безопасный *REST API* сервиса планирования задач по методу *GTD*. Реализованы регистрация и вход с *JWT*, ролевая модель *user*, *moderator* и *admin* с проверкой роли в *middleware* и проверкой владельца в сервисах, хранение данных в *PostgreSQL* через *Sequelize*, проверка всех входных данных схемами *Joi*, глобальный лимит запросов и журнал подозрительных действий.

Дополнительно реализованы десять мер: сложность пароля, *refresh*-токен в *httpOnly cookie* с ротацией, блокировка после пяти неудачных входов, защита от *mass assignment*, лимит тела 10 КБ, приём только *application/json*, санитизация *HTML*, *CORS* по белому списку, ошибки без деталей в *production* и журнал ошибок с ротацией. Каждая мера проверена реальным запросом к серверу, результаты приведены в разделе 2.

Коллекция *Postman* из 102 запросов и 184 проверок выполнена через *Newman* без ошибок, а ручная проверка дополнительных мер подтвердила их работу на реальных запросах.

Код размещён в репозитории <https://github.com/Alexandr-Obraztsov/gtd-task-service> в ветке *ipr21*, коллекция *Postman* — по адресу https://github.com/Alexandr-Obraztsov/gtd-task-service/blob/ipr21/postman/gtd-secure-api.postman_collection.json. Разработанный *API* служит серверной частью курсового проекта и будет дополнен клиентским приложением и рассылкой напоминаний.